

CO4-AT1 Industrial problem solving

CSA0904-Programming in Java

Kandath Haneesh 192324084

Ticket Booking System

```
import javax.swing.*;
import java.awt.*;
import java.util.LinkedHashMap;
import java.util.Map;

public class TicketBookingSystem extends JFrame {

    // Stores seat number and passenger name
    static Map<Integer, String> bookings = new LinkedHashMap<>();
    static final int TOTAL_SEATS = 20;

    JLabel statusLabel;

    public TicketBookingSystem()
    { setTitle("Ticket Booking System");
      setSize(600, 420);
      setLocationRelativeTo(null);
      setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

      createMenu();
```

```
JLabel title = new JLabel("TICKET BOOKING SYSTEM",  
SwingConstants.CENTER);
```

```
title.setFont(new Font("Arial", Font.BOLD, 24));
```

```
JButton bookButton = new JButton("Book Ticket");
```

```
JButton viewButton = new JButton("View Bookings");
```

```
JButton cancelButton = new JButton("Cancel Ticket");
```

```
JButton exitButton = new JButton("Exit");
```

```
bookButton.addActionListener(e -> bookTicket());
```

```
viewButton.addActionListener(e -> viewBookings());
```

```
cancelButton.addActionListener(e -> cancelTicket());
```

```
exitButton.addActionListener(e -> System.exit(0));
```

```
JPanel buttons = new JPanel(new GridLayout(2, 2, 15, 15));
```

```
buttons.add(bookButton);
```

```
buttons.add(viewButton);
```

```
buttons.add(cancelButton);
```

```
buttons.add(exitButton);
```

```
statusLabel = new JLabel("", SwingConstants.CENTER);
```

```
JPanel panel = new JPanel(new BorderLayout(15, 15));
```

```
panel.setBorder(BorderFactory.createEmptyBorder(25, 25, 25, 25));
```

```
panel.add(title, BorderLayout.NORTH);
```

```

panel.add(buttons, BorderLayout.CENTER);
panel.add(statusLabel, BorderLayout.SOUTH);

add(panel);
updateStatus();
}

// Create menu
void createMenu() {
    JMenuBar bar = new JMenuBar();
    JMenu menu = new JMenu("Booking");

    JMenuItem book = new JMenuItem("Book Ticket");
    JMenuItem view = new JMenuItem("View Bookings");
    JMenuItem cancel = new JMenuItem("Cancel Ticket");
    JMenuItem exit = new JMenuItem("Exit");

    book.addActionListener(e -> bookTicket());
    view.addActionListener(e -> viewBookings());
    cancel.addActionListener(e -> cancelTicket());
    exit.addActionListener(e -> System.exit(0));

    menu.add(book);
    menu.add(view);
    menu.add(cancel);
    menu.addSeparator();
}

```

```
menu.add(exit);

bar.add(menu);
setJMenuBar(bar);
}

// Book a ticket
void bookTicket() {
    String name =
        JOptionPane.showInputDialog( this,
            "Enter passenger name:");

    if (name == null) return;

    name = name.trim();

    if (name.isEmpty())
    { JOptionPane.showMessageDialog(this,
        "Passenger name cannot be empty.");
        return;
    }

    if (!name.matches("[a-zA-Z ]+"))
    { JOptionPane.showMessageDialog(this,
        "Enter a valid name using letters only.");
        return;
    }
}
```

```

String input =
    JOptionPane.showInputDialog( this,
        getAvailableSeats() +
        "\nEnter seat number (1-20):");

if (input == null) return;

try {
    int seat = Integer.parseInt(input);

    if (seat < 1 || seat > TOTAL_SEATS)
        { throw new IllegalArgumentException(
            "Seat number must be between 1 and 20.");
        }

    if (bookings.containsKey(seat))
        { JOptionPane.showMessageDialog(this,
            "Seat " + seat + " is already booked.");
        return;
        }

    bookings.put(seat, name);

    JOptionPane.showMessageDialog(this,
        "Booking successful!\n\nPassenger: " +
        name + "\nSeat: " + seat);
}

```

```

        updateStatus();

    } catch (NumberFormatException ex)
    { JOptionPane.showMessageDialog(this,
        "Please enter a valid number.");
    } catch (IllegalArgumentException ex)
    { JOptionPane.showMessageDialog(this,
        ex.getMessage());
    }
}

// Display bookings
void viewBookings() {
    if (bookings.isEmpty())
    { JOptionPane.showMessageDialog(this,
        "No bookings found.");
        return;
    }

    StringBuilder text = new StringBuilder();
    text.append("CONFIRMED BOOKINGS\n");
    text.append("=====\n\n");

    for (Map.Entry<Integer, String> entry : bookings.entrySet())
    { text.append("Seat ")

```

```
.append(entry.getKey())
.append(" -> ")
.append(entry.getValue())
.append("\n");
}
```

```
text.append("\nBooked: ").append(bookings.size());
text.append("\nAvailable: ").append(
    TOTAL_SEATS - bookings.size());
```

```
JOptionPane.showMessageDialog(this, text.toString(),
    "Booking Confirmation",
    JOptionPane.INFORMATION_MESSAGE);
}
```

```
// Cancel a ticket
```

```
void cancelTicket() {
```

```
    String input =
```

```
        JOptionPane.showInputDialog( this, "Enter
        seat number to cancel (1-20):");
```

```
    if (input == null) return;
```

```
    try {
```

```
        int seat = Integer.parseInt(input);
```

```
        if (seat < 1 || seat > TOTAL_SEATS) {
```

```

        throw new IllegalArgumentException(
            "Seat number must be between 1 and 20.");
    }

    if (!bookings.containsKey(seat))
    { JOptionPane.showMessageDialog(this,
        "This seat is not booked.");
        return;
    }

    String passenger = bookings.get(seat);

    int choice = JOptionPane.showConfirmDialog(this,
        "Cancel booking for " + passenger +
        " at Seat " + seat + "?",
        "Confirm Cancellation",
        JOptionPane.YES_NO_OPTION);

    if (choice == JOptionPane.YES_OPTION)
    { bookings.remove(seat);

        JOptionPane.showMessageDialog(this,
            "Ticket cancelled successfully.");

        updateStatus();
    }

```

```

    } catch (NumberFormatException ex)
    { JOptionPane.showMessageDialog(this,
        "Please enter a valid number.");
    } catch (IllegalArgumentException ex)
    { JOptionPane.showMessageDialog(this,
        ex.getMessage());
    }
}

```

```

String getAvailableSeats() {
    StringBuilder seats = new StringBuilder("Available seats:\n");

    for (int i = 1; i <= TOTAL_SEATS; i++)
    { if (!bookings.containsKey(i)) {
        seats.append(i).append(" ");
    }
    }

    return seats.toString();
}

```

```

void updateStatus()
{ statusLabel.setText
(
    "Total Seats: " + TOTAL_SEATS +
    " | Booked: " + bookings.size() +

```

```

        " | Available: " +
        (TOTAL_SEATS - bookings.size()));
    }

    public static void main(String[] args) { SwingUtilities.invokeLater(()
    -> {
        new TicketBookingSystem().setVisible(true);
    });
    }
}

```

Output:





